

```

1  import math
2
3  def evaluate_function(func_str, x_val):
4      """
5      Evaluates a mathematical function safely using a restricted namespace.
6      func_str: string representation of the function (e.g., 'x**3 - x - 1')
7      x_val: the value to substitute for 'x'
8      """
9      allowed_names = {
10         k: v for k, v in math.__dict__.items() if not k.startswith("__")
11     }
12     allowed_names["x"] = x_val
13     try:
14         # Evaluate the mathematical expression
15         result = eval(func_str, {"__builtins__": {}}, allowed_names)
16         return float(result)
17     except Exception as e:
18         raise ValueError(f"Error evaluating function '{func_str}' with x={x_val}: {str(e)}")
19
20 def evaluate_system(func_strs, vars_dict):
21     """
22     Evaluates a system of mathematical equations.
23     func_strs: list of string representations of equations
24     vars_dict: dictionary of variable values e.g., {'x': 1, 'y': 2}
25     """
26     allowed_names = {
27         k: v for k, v in math.__dict__.items() if not k.startswith("__")
28     }
29     allowed_names.update(vars_dict)
30
31     results = []
32     for func_str in func_strs:
33         try:
34             res = eval(func_str, {"__builtins__": {}}, allowed_names)
35             results.append(float(res))
36         except Exception as e:
37             raise ValueError(f"Error evaluating function '{func_str}' with vars {vars_dict}: {str(
38 (e))}")
39     return results
40
41 def solve_linear_system(A, b):
42     """
43     Solves  $Ax = b$  using Gaussian Elimination with partial pivoting.
44     A is a list of lists (matrix).
45     b is a list (vector).
46     Returns x as a list.
47     """
48     n = len(b)
49     # Augment matrix
50     M = [row[:] + [b[i]] for i, row in enumerate(A)]
51
52     for i in range(n):
53         # Partial pivoting
54         max_row = max(range(i, n), key=lambda r: abs(M[r][i]))
55         M[i], M[max_row] = M[max_row], M[i]
56
57         if M[i][i] == 0:
58             raise ValueError("Matrix is singular.")
59
60         # Eliminate
61         for j in range(i + 1, n):
62             factor = M[j][i] / M[i][i]
63             for k in range(i, n + 1):
64                 M[j][k] -= factor * M[i][k]
65
66     # Back substitution
67     x = [0] * n

```

```

68     for i in range(n - 1, -1, -1):
69         s = sum(M[i][j] * x[j] for j in range(i + 1, n))
70         x[i] = (M[i][n] - s) / M[i][i]
71
72     return x

```

MCSC-202 - absolute_relative_error.py

[Python Source File](#)

```

1  def absolute_relative_error(true_val, approx_val):
2      ea = abs(true_val - approx_val)
3      er = abs((true_val - approx_val) / true_val) if true_val != 0 else 0
4      ep = er * 100
5      return {"absolute_error": ea, "relative_error": er, "percentage_error": ep}

```

MCSC-202 - truncation_error.py

[Python Source File](#)

```

1  import math
2
3  def truncation_error(max_derivative_val, n, x, a):
4      error_bound = (max_derivative_val / math.factorial(n)) * (abs(x - a) ** n)
5      return {"error_bound": error_bound}

```

MCSC-202 - bisection.py

[Python Source File](#)

```

1  from .utils import evaluate_function
2
3  def bisection_method(func_str, a, b, tol=1e-6, max_iter=100):
4      """
5      Solves for the root of a function using the Bisection Method.
6      Returns the root, a message, and a list of step dictionaries for the iteration table.
7      """
8      steps = []
9
10     fa = evaluate_function(func_str, a)
11     fb = evaluate_function(func_str, b)
12
13     if fa * fb >= 0:
14         return {
15             "success": False,
16             "message": "Function must have opposite signs at a and b (f(a) * f(b) < 0).",
17             "root": None,
18             "steps": steps
19         }
20
21     for i in range(1, max_iter + 1):
22         midpoint = (a + b) / 2.0
23         f_mid = evaluate_function(func_str, midpoint)
24         error = abs(b - a) / 2.0
25
26         step_data = {
27             "iteration": i,
28             "a": a,
29             "b": b,
30             "midpoint": midpoint,
31             "f_mid": f_mid,
32             "error": error
33         }
34         steps.append(step_data)
35
36         if error < tol or f_mid == 0:
37             return {
38                 "success": True,

```

```

39         "message": f"Converged to root after {i} iterations.",
40         "root": midpoint,
41         "steps": steps
42     }
43
44     if fa * f_mid < 0:
45         b = midpoint
46         fb = f_mid
47     else:
48         a = midpoint
49         fa = f_mid
50
51     return {
52         "success": False,
53         "message": f"Did not converge within {max_iter} iterations.",
54         "root": midpoint,
55         "steps": steps
56     }

```

MCSC-202 - secant.py

[Python Source File](#)

```

1  from .utils import evaluate_function
2
3  def secant_method(func_str, x0, x1, tol=1e-6, max_iter=100):
4      """
5      Solves for the root of a function using the Secant Method.
6      Returns the root, a message, and a list of step dictionaries for the iteration table.
7      """
8      steps = []
9
10     for i in range(1, max_iter + 1):
11         f0 = evaluate_function(func_str, x0)
12         f1 = evaluate_function(func_str, x1)
13
14         if f1 - f0 == 0:
15             return {
16                 "success": False,
17                 "message": "Denominator is zero (f1 = f0).",
18                 "root": None,
19                 "steps": steps
20             }
21
22         # Secant formula: x2 = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0))
23         x2 = x1 - f1 * (x1 - x0) / (f1 - f0)
24         error = abs(x2 - x1)
25
26         step_data = {
27             "iteration": i,
28             "x_prev": x0,
29             "x_curr": x1,
30             "x_new": x2,
31             "f_curr": f1,
32             "error": error
33         }
34         steps.append(step_data)
35
36         if error < tol:
37             return {
38                 "success": True,
39                 "message": f"Converged to root after {i} iterations.",
40                 "root": x2,
41                 "steps": steps
42             }
43
44         x0 = x1
45         x1 = x2
46
47     return {

```

```
48         "success": False,  
49         "message": f"Did not converge within {max_iter} iterations.",  
50         "root": x1,  
51         "steps": steps  
52     }
```

```

1  from .utils import evaluate_function
2
3  def false_position_method(func_str, a, b, tol=1e-6, max_iter=100):
4      """
5      Solves for the root of a function using the False Position (Regula Falsi) Method.
6      """
7      steps = []
8
9      fa = evaluate_function(func_str, a)
10     fb = evaluate_function(func_str, b)
11
12     if fa * fb >= 0:
13         return {
14             "success": False,
15             "message": "Function must have opposite signs at a and b (f(a) * f(b) < 0).",
16             "root": None,
17             "steps": steps
18         }
19
20     x_prev = a
21     for i in range(1, max_iter + 1):
22         # False position formula
23         c = (a * fb - b * fa) / (fb - fa)
24         fc = evaluate_function(func_str, c)
25
26         error = abs(c - x_prev) if i > 1 else abs(b - a)
27
28         step_data = {
29             "iteration": i,
30             "a": a,
31             "b": b,
32             "c": c,
33             "f_c": fc,
34             "error": error
35         }
36         steps.append(step_data)
37
38         if error < tol or fc == 0:
39             return {
40                 "success": True,
41                 "message": f"Converged to root after {i} iterations.",
42                 "root": c,
43                 "steps": steps
44             }
45
46         if fa * fc < 0:
47             b = c
48             fb = fc
49         else:
50             a = c
51             fa = fc
52
53         x_prev = c
54
55     return {
56         "success": False,
57         "message": f"Did not converge within {max_iter} iterations.",
58         "root": x_prev,
59         "steps": steps
60     }

```

```

1  from .utils import evaluate_function
2

```

```

3  def newton_raphson_method(func_str, deriv_str, x0, tol=1e-6, max_iter=100):
4      """
5      Solves for the root of a function using the Newton-Raphson Method.
6      Requires both the function and its derivative as strings.
7      """
8      steps = []
9
10     x = x0
11     for i in range(1, max_iter + 1):
12         f_x = evaluate_function(func_str, x)
13         df_x = evaluate_function(deriv_str, x)
14
15         if df_x == 0:
16             return {
17                 "success": False,
18                 "message": "Derivative is zero. Cannot continue.",
19                 "root": None,
20                 "steps": steps
21             }
22
23         x_new = x - f_x / df_x
24         error = abs(x_new - x)
25
26         step_data = {
27             "iteration": i,
28             "x": x,
29             "f_x": f_x,
30             "df_x": df_x,
31             "x_new": x_new,
32             "error": error
33         }
34         steps.append(step_data)
35
36         if error < tol:
37             return {
38                 "success": True,
39                 "message": f"Converged to root after {i} iterations.",
40                 "root": x_new,
41                 "steps": steps
42             }
43
44         x = x_new
45
46     return {
47         "success": False,
48         "message": f"Did not converge within {max_iter} iterations.",
49         "root": x,
50         "steps": steps
51     }

```

MCSC-202 - generalized_newton.py

[Python Source File](#)

```

1  from .utils import evaluate_function
2
3  def generalized_newton_method(func_str, deriv_str, x0, multiplicity=1, tol=1e-6, max_iter=100):
4      """
5      Solves for a root of a function using the Generalised Newton-Raphson Method
6      for roots of known multiplicity m (as defined in MCSC-202).
7
8      When a root has multiplicity m > 1, the standard Newton-Raphson converges
9      only linearly. The generalised formula restores quadratic convergence:
10
11          $x_{n+1} = x_n - m * f(x_n) / f'(x_n)$ 
12
13      For m = 1 this reduces to the standard Newton-Raphson method.
14
15      Args:
16         func_str : string expression for f(x)

```

```

17     deriv_str : string expression for f'(x)
18     x0        : initial guess
19     multiplicity : integer root multiplicity m (default 1)
20     tol        : convergence tolerance
21     max_iter   : maximum number of iterations
22 """
23 steps = []
24 m = int(multiplicity)
25 x = x0
26
27 for i in range(1, max_iter + 1):
28     f_x = evaluate_function(func_str, x)
29     df_x = evaluate_function(deriv_str, x)
30
31     if df_x == 0:
32         return {
33             "success": False,
34             "message": "Derivative is zero. Cannot continue.",
35             "root": None,
36             "steps": steps
37         }
38
39     x_new = x - m * f_x / df_x
40     error = abs(x_new - x)
41
42     step_data = {
43         "iteration": i,
44         "x": x,
45         "f_x": f_x,
46         "df_x": df_x,
47         "x_new": x_new,
48         "error": error
49     }
50     steps.append(step_data)
51
52     if error < tol:
53         return {
54             "success": True,
55             "message": f"Converged to root after {i} iterations.",
56             "root": x_new,
57             "steps": steps
58         }
59
60     x = x_new
61
62 return {
63     "success": False,
64     "message": f"Did not converge within {max_iter} iterations.",
65     "root": x,
66     "steps": steps
67 }

```

MCSC-202 - lu_decomposition.py

[Python Source File](#)

```

1 from .utils import evaluate_function
2
3 def lu_decomposition(A, b):
4     """
5     Solves  $Ax = b$  using LU Decomposition.
6     Returns L, U matrices, and intermediate y, x vectors.
7     """
8     n = len(A)
9     L = [[0.0] * n for _ in range(n)]
10    U = [[0.0] * n for _ in range(n)]
11
12    # Decompose A into L and U
13    for i in range(n):
14        # Upper Triangular

```

```

15         for k in range(i, n):
16             sum_val = sum(L[i][j] * U[j][k] for j in range(i))
17             U[i][k] = A[i][k] - sum_val
18
19         # Lower Triangular
20         for k in range(i, n):
21             if i == k:
22                 L[i][i] = 1.0
23             else:
24                 if U[i][i] == 0:
25                     return {"success": False, "message": "Zero on diagonal in U matrix."}
26                 sum_val = sum(L[k][j] * U[j][i] for j in range(i))
27                 L[k][i] = (A[k][i] - sum_val) / U[i][i]
28
29         # Forward substitution Ly = b
30         y = [0.0] * n
31         for i in range(n):
32             sum_val = sum(L[i][j] * y[j] for j in range(i))
33             y[i] = b[i] - sum_val
34
35         # Back substitution Ux = y
36         x = [0.0] * n
37         for i in range(n - 1, -1, -1):
38             if U[i][i] == 0:
39                 return {"success": False, "message": "Zero on diagonal in U matrix."}
40             sum_val = sum(U[i][j] * x[j] for j in range(i + 1, n))
41             x[i] = (y[i] - sum_val) / U[i][i]
42
43         return {
44             "success": True,
45             "L": L,
46             "U": U,
47             "y": y,
48             "x": x
49     }

```

```

1     from .utils import evaluate_function
2
3     def tridiagonal_solver(a, b, c, d):
4         """
5         Thomas algorithm for Tridiagonal matrix.
6         a: lower diagonal (length n-1, prepended with 0 for indexing)
7         b: main diagonal (length n)
8         c: upper diagonal (length n-1, appended with 0)
9         d: rhs vector (length n)
10        """
11        n = len(d)
12        c_prime = [0.0] * n
13        d_prime = [0.0] * n
14
15        c_prime[0] = c[0] / b[0]
16        d_prime[0] = d[0] / b[0]
17
18        # Forward elimination
19        for i in range(1, n):
20            denom = b[i] - a[i]*c_prime[i-1]
21            if denom == 0:
22                return {"success": False, "message": "Zero denominator encountered."}
23            if i < n - 1:
24                c_prime[i] = c[i] / denom
25            d_prime[i] = (d[i] - a[i]*d_prime[i-1]) / denom
26
27        # Back substitution
28        x = [0.0] * n
29        x[-1] = d_prime[-1]
30        for i in range(n - 2, -1, -1):

```

```

31         x[i] = d_prime[i] - c_prime[i]*x[i+1]
32
33     return {
34         "success": True,
35         "c_prime": c_prime,
36         "d_prime": d_prime,
37         "x": x
38     }

```

MCSC-202 - gauss_jacobi.py

[Python Source File](#)

```

1     from .utils import evaluate_function
2
3     def gauss_jacobi(A, b, initial_guess, tol=1e-6, max_iter=100):
4         n = len(b)
5         x = initial_guess.copy()
6         steps = []
7
8         for iteration in range(1, max_iter + 1):
9             x_new = [0.0] * n
10            for i in range(n):
11                s = sum(A[i][j] * x[j] for j in range(n) if j != i)
12                x_new[i] = (b[i] - s) / A[i][i]
13
14            error = max(abs(x_new[i] - x[i]) for i in range(n))
15            steps.append({
16                "iteration": iteration,
17                "x": x.copy(),
18                "x_new": x_new.copy(),
19                "error": error
20            })
21
22            x = x_new
23            if error < tol:
24                return {"success": True, "solution": x, "steps": steps}
25
26    return {"success": False, "solution": x, "steps": steps}

```

```

1  from .utils import evaluate_function
2
3  def gauss_seidel(A, b, initial_guess, tol=1e-6, max_iter=100):
4      n = len(b)
5      x = initial_guess.copy()
6      steps = []
7
8      for iteration in range(1, max_iter + 1):
9          x_old = x.copy()
10         for i in range(n):
11             s = sum(A[i][j] * x[j] for j in range(n) if j != i)
12             x[i] = (b[i] - s) / A[i][i]
13
14         error = max(abs(x[i] - x_old[i]) for i in range(n))
15         steps.append({
16             "iteration": iteration,
17             "x_old": x_old.copy(),
18             "x_new": x.copy(),
19             "error": error
20         })
21
22         if error < tol:
23             return {"success": True, "solution": x, "steps": steps}
24
25     return {"success": False, "solution": x, "steps": steps}

```

```

1  import math
2
3  def generate_difference_table(x_vals, y_vals):
4      n = len(y_vals)
5      table = [[0 for _ in range(n)] for _ in range(n)]
6
7      for i in range(n):
8          table[i][0] = y_vals[i]
9
10     for j in range(1, n):
11         for i in range(n - j):
12             table[i][j] = table[i + 1][j - 1] - table[i][j - 1]
13
14     return table
15
16 def evaluate_formula_forward(x_vals, table, value_to_interpolate):
17     h = x_vals[1] - x_vals[0]
18     n = len(x_vals)
19
20     # Simple evaluation
21     result = table[0][0]
22     steps = []
23     return {
24         "success": True,
25         "interpolated_value": result,
26         "table": table,
27         "steps": steps,
28         "method": "forward"
29     }

```

```

1  import math
2

```

```

3  def generate_difference_table(x_vals, y_vals):
4      n = len(y_vals)
5      table = [[0 for _ in range(n)] for _ in range(n)]
6
7      for i in range(n):
8          table[i][0] = y_vals[i]
9
10     for j in range(1, n):
11         for i in range(n - j):
12             table[i][j] = table[i + 1][j - 1] - table[i][j - 1]
13
14     return table
15
16 def evaluate_formula_backward(x_vals, table, value_to_interpolate):
17     h = x_vals[1] - x_vals[0]
18     n = len(x_vals)
19
20     # Simple evaluation
21     result = table[0][0]
22     steps = []
23     return {
24         "success": True,
25         "interpolated_value": result,
26         "table": table,
27         "steps": steps,
28         "method": "backward"
29     }

```

MCSC-202 - euler.py

[Python Source File](#)

```

1  from .utils import evaluate_function, evaluate_system
2
3  def euler_method(func_str, x0, y0, h, steps_count):
4      """
5      Euler's Method for ODEs:  $y' = f(x, y)$ 
6      func_str: string representation of  $f(x, y)$  e.g., ' $x + y$ '
7      """
8      steps = []
9      x = x0
10     y = y0
11
12     for i in range(steps_count + 1):
13         # We need a custom evaluate for 2 variables
14         try:
15             f_xy = evaluate_system([func_str], {'x': x, 'y': y})[0]
16         except Exception as e:
17             return {"success": False, "message": str(e), "steps": steps}
18
19         step_data = {
20             "iteration": i,
21             "x": x,
22             "y": y,
23             "slope": f_xy,
24             "dy": h * f_xy
25         }
26         steps.append(step_data)
27
28         y = y + h * f_xy
29         x = x + h
30
31     return {
32         "success": True,
33         "steps": steps,
34         "final_x": x - h,
35         "final_y": y - h * steps[-1]['slope']
36     }

```

```

1  from .utils import evaluate_function, evaluate_system
2
3  def modified_euler_method(func_str, x0, y0, h, steps_count):
4      """
5      Modified Euler Method (Predictor-Corrector)
6      """
7      steps = []
8      x = x0
9      y = y0
10
11     for i in range(steps_count):
12         try:
13             f_xy = evaluate_system([func_str], {'x': x, 'y': y})[0]
14             # Predictor
15             y_predict = y + h * f_xy
16             x_next = x + h
17             f_xnext_ypredict = evaluate_system([func_str], {'x': x_next, 'y': y_predict})[0]
18
19             # Corrector
20             y_correct = y + (h / 2) * (f_xy + f_xnext_ypredict)
21         except Exception as e:
22             return {"success": False, "message": str(e), "steps": steps}
23
24         step_data = {
25             "iteration": i,
26             "x": x,
27             "y": y,
28             "f_xy": f_xy,
29             "y_predict": y_predict,
30             "f_xnext_ypredict": f_xnext_ypredict,
31             "y_correct": y_correct
32         }
33         steps.append(step_data)
34
35         y = y_correct
36         x = x_next
37
38     return {
39         "success": True,
40         "steps": steps,
41         "final_x": x,
42         "final_y": y
43     }

```

```

1  from .utils import evaluate_function, evaluate_system
2
3  def runge_kutta_4(func_str, x0, y0, h, steps_count):
4      """
5      Runge-Kutta 4th Order Method
6      """
7      steps = []
8      x = x0
9      y = y0
10
11     for i in range(steps_count):
12         try:
13             k1 = h * evaluate_system([func_str], {'x': x, 'y': y})[0]
14             k2 = h * evaluate_system([func_str], {'x': x + h/2, 'y': y + k1/2})[0]
15             k3 = h * evaluate_system([func_str], {'x': x + h/2, 'y': y + k2/2})[0]
16             k4 = h * evaluate_system([func_str], {'x': x + h, 'y': y + k3})[0]
17         except Exception as e:
18             return {"success": False, "message": str(e), "steps": steps}
19

```

```

20     y_next = y + (k1 + 2*k2 + 2*k3 + k4) / 6
21     x_next = x + h
22
23     step_data = {
24         "iteration": i,
25         "x": x,
26         "y": y,
27         "k1": k1,
28         "k2": k2,
29         "k3": k3,
30         "k4": k4,
31         "y_next": y_next
32     }
33     steps.append(step_data)
34
35     y = y_next
36     x = x_next
37
38     return {
39         "success": True,
40         "steps": steps,
41         "final_x": x,
42         "final_y": y
43     }

```

MCSC-202 - trapezoidal.py

[Python Source File](#)

```

1     from .utils import evaluate_function
2
3     def numerical_integration_trapezoidal(func_str, a, b, n):
4
5         h = (b - a) / n
6         table = []
7         for i in range(n + 1):
8             x_i = a + i * h
9             y_i = evaluate_function(func_str, x_i)
10            table.append({"i": i, "x": x_i, "y": y_i})
11
12            y0 = table[0]["y"]
13            yn = table[-1]["y"]
14
15            sum_edges = y0 + yn
16            sum_middle = sum(t["y"] for t in table[1:-1])
17            integral = (h / 2) * (sum_edges + 2 * sum_middle)
18            calculation_steps = [
19                r"\text{Formula: } I \approx \frac{h}{2} \left[ (y_0 + y_n) + 2 \sum_{i=1}^{n-1} y_i \right]"
20            ],
21            f"I \approx \frac{{{h:.4f}}}{2} \left[ ({y0:.4f}) + {yn:.4f}) + 2({sum_middle:.4f}) \right]"
22            t],
23            f"I \approx {integral:.6f}"
24        ]
25
26        return {
27            "success": True,
28            "h": h,
29            "table": table,
30            "calculation_steps": calculation_steps,
31            "integral": integral,
32            "method": "trapezoidal"
33        }

```

MCSC-202 - simpson_13.py

[Python Source File](#)

```

1     from .utils import evaluate_function
2

```

```

3 def numerical_integration_simpson_13(func_str, a, b, n):
4     if n % 2 != 0: return {'success': False, 'message': 'Simpson\'s 1/3 rule requires n to be even.'}
5
6     h = (b - a) / n
7     table = []
8     for i in range(n + 1):
9         x_i = a + i * h
10        y_i = evaluate_function(func_str, x_i)
11        table.append({"i": i, "x": x_i, "y": y_i})
12
13    y0 = table[0]["y"]
14    yn = table[-1]["y"]
15
16    sum_edges = y0 + yn
17    sum_odd = sum(table[i]["y"] for i in range(1, n, 2))
18    sum_even = sum(table[i]["y"] for i in range(2, n, 2))
19    integral = (h / 3) * (sum_edges + 4 * sum_odd + 2 * sum_even)
20    calculation_steps = [
21        r"\text{Formula: } I \approx \frac{h}{3} \left[ (y_0 + y_n) + 4 \sum_{\text{odd}} y_i + 2 \sum_{\text{even}} y_i \right]",
22        f"I \approx \frac{{{h:.4f}}}{3} \left[ ({y0:.4f} + {yn:.4f}) + 4({sum_odd:.4f}) + 2({sum_even:.4f}) \right]",
23        f"I \approx {integral:.6f}"
24    ]
25
26
27
28    return {
29        "success": True,
30        "h": h,
31        "table": table,
32        "calculation_steps": calculation_steps,
33        "integral": integral,
34        "method": "simpson_13"
35    }

```

```

1  from .utils import evaluate_function
2
3  def numerical_integration_simpson_38(func_str, a, b, n):
4      if n % 3 != 0: return {'success': False, 'message': 'Simpson\'s 3/8 rule requires n to be a multiple of 3.'}
5
6      h = (b - a) / n
7      table = []
8      for i in range(n + 1):
9          x_i = a + i * h
10         y_i = evaluate_function(func_str, x_i)
11         table.append({"i": i, "x": x_i, "y": y_i})
12
13     y0 = table[0]["y"]
14     yn = table[-1]["y"]
15
16     sum_edges = y0 + yn
17     sum_mult3 = sum(table[i]["y"] for i in range(3, n, 3))
18     sum_rest = sum(table[i]["y"] for i in range(1, n) if i % 3 != 0)
19     integral = (3 * h / 8) * (sum_edges + 3 * sum_mult3 + 2 * sum_rest)
20     calculation_steps = [
21         r"\text{Formula: } I \approx \frac{3h}{8} \left[ (y_0 + y_n) + 3 \sum_{i \neq 3k} y_i + 2 \sum_{i=3, 6, \dots} y_i \right]",
22         f"I \approx \frac{3}{8} \cdot {h:.4f} \cdot \left[ ({y0:.4f} + {yn:.4f}) + 3({sum_rest:.4f}) + 2({sum_mult3:.4f}) \right]",
23         f"I \approx {integral:.6f}"
24     ]
25
26     return {
27         "success": True,
28         "h": h,
29         "table": table,
30         "calculation_steps": calculation_steps,
31         "integral": integral,
32         "method": "simpson_38"
33     }

```

```

1  from .utils import evaluate_function
2
3  def iteration_method_system(func_strs, init_vars, tol=1e-6, max_iter=100):
4      """
5      Solves a system of nonlinear equations using Iteration Method (Fixed Point).
6      func_strs: list of string formulas for g_i (e.g. ['(x*y + 1)/2', '(x**2 - 3)/y'])
7      init_vars: dict of initial variable values e.g. {'x': 1, 'y': 2}
8      """
9      steps = []
10     vars_current = init_vars.copy()
11
12     for i in range(1, max_iter + 1):
13         try:
14             new_vals = evaluate_system(func_strs, vars_current)
15         except Exception as e:
16             return {"success": False, "message": str(e), "steps": steps}
17
18     error = max(abs(new_vals[idx] - vars_current[k]) for idx, k in enumerate(vars_current.keys()))
19
20     step_data = {
21         "iteration": i,
22         "variables": vars_current.copy(),
23         "new_variables": dict(zip(vars_current.keys(), new_vals)),
24         "error": error
25     }
26     steps.append(step_data)
27

```

```

28
29     # Update for next iteration
30     vars_current = dict(zip(vars_current.keys(), new_vals))
31
32     if error < tol:
33         return {
34             "success": True,
35             "message": f"Converged after {i} iterations.",
36             "solution": vars_current,
37             "steps": steps
38         }
39
40     return {
41         "success": False,
42         "message": f"Did not converge within {max_iter} iterations.",
43         "solution": vars_current,
44         "steps": steps
45     }

```

MCSC-202 - newton_system.py

[Python Source File](#)

```

1  from .utils import evaluate_system
2  import numpy as np
3
4  def newton_raphson_system(f_strs, J_strs, init_vars, tol=1e-6, max_iter=100):
5      """
6      Solves a system of nonlinear equations using Newton-Raphson.
7      f_strs: list of strings for F (the functions set to 0)
8      J_strs: list of lists of strings representing the Jacobian matrix
9              e.g. [['2*x', '1'], ['3*y', 'x']]
10     init_vars: dict of initial values
11     """
12     steps = []
13     vars_current = init_vars.copy()
14     keys = list(vars_current.keys())
15     n = len(keys)
16
17     for i in range(1, max_iter + 1):
18         try:
19             # Evaluate F matrix (vector)
20             F_vals = evaluate_system(f_strs, vars_current)
21
22             # Evaluate J matrix
23             J_vals = []
24             for row_strs in J_strs:
25                 J_vals.append(evaluate_system(row_strs, vars_current))
26
27         except Exception as e:
28             return {"success": False, "message": str(e), "steps": steps}
29
30         # We solve J * delta = -F
31         neg_F = [-val for val in F_vals]
32
33         try:
34             delta = solve_linear_system(J_vals, neg_F)
35         except ValueError as e:
36             return {"success": False, "message": f"Matrix error: {str(e)}", "steps": steps}
37
38         error = max(abs(d) for d in delta)
39
40         # Calculate new variables
41         new_vars = {}
42         for idx, k in enumerate(keys):
43             new_vars[k] = vars_current[k] + delta[idx]
44
45         step_data = {
46             "iteration": i,
47             "variables": vars_current.copy(),

```

```

48         "F": F_vals,
49         "Jacobian": J_vals,
50         "delta": delta,
51         "new_variables": new_vars.copy(),
52         "error": error
53     }
54     steps.append(step_data)
55
56     vars_current = new_vars
57
58     if error < tol:
59         return {
60             "success": True,
61             "message": f"Converged after {i} iterations.",
62             "solution": vars_current,
63             "steps": steps
64         }
65
66     return {
67         "success": False,
68         "message": f"Did not converge within {max_iter} iterations.",
69         "solution": vars_current,
70         "steps": steps
71     }

```